Java Design Patterns — Interview Master Notes

Part 1: Fundamentals + Singleton + Factory Method

Personal Interview Prep Notes

2026

Contents

PART 1 — DESIGN PATTERN FUNDAMENTALS **3**

1. What is a Design Pattern? [BEG] . . . 3
2. Why were Design Patterns Introduced? [BEG] . . . 3
3. What Problem Do Design Patterns Solve? [BEG] . . . 3
4. Design Pattern vs Framework [BEG] . . . 3
5. Design Pattern vs Library [BEG] . . . 4
6. Design Pattern vs Algorithm [BEG] . . . 4
7. Design Pattern vs Architecture [INT] . . . 4
8. Advantages of Design Patterns [BEG] . . . 4
9. Disadvantages of Design Patterns [INT] . . . 4
10. What is GoF? [BEG] . . . 4
11. The 23 GoF Design Patterns [BEG] . . . 4
12. The Three Major Categories [BEG] . . . 5
13. How Design Patterns Relate to OOP [INT] . . . 5
14. Design Patterns vs SOLID Principles [ADV] . . . 5
15. Can Design Patterns Violate SOLID? [ADV] (Tricky Interview Question) . . . 6
16. Should Every Application Use Design Patterns? [INT] (Common Interview Question) 6

PART 2 — CREATIONAL DESIGN PATTERNS **7**

1. Singleton Pattern [BEG] **7**

1.1 Definition . . . 7
1.2 The Problem (before Singleton) . . . 7
1.3 Why Singleton? — What It Solves . . . 7
1.4 Step-by-Step Working . . . 7
1.5 Class Relationship Diagram . . . 7
1.6 Eager Initialization Singleton . . . 7
1.7 Lazy Initialization Singleton (Not Thread-Safe) . . . 8
1.8 Thread-Safe Singleton — `synchronized` method [INT] . . . 8
1.9 Double-Checked Locking + `volatile` [ADV] (Most Asked in Interviews) . . . 9
1.10 Enum Singleton [ADV] (Best Practice — Josh Bloch recommended) . . . 9
1.11 The Serialization Problem [ADV] . . . 10
1.12 The Reflection Problem [ADV] . . . 10
1.13 The Cloning Problem [INT] . . . 10
1.14 Singleton vs Static Class [INT] (Very Common Interview Question) . . . 11
1.15 Singleton in Spring Boot [ADV] (Very Important — Common Confusion) . . . 11
1.16 Real-World Examples . . . 12
1.17 Best Implementation in Modern Java (Java 17/21) [ADV] . . . 12
1.18 Advantages [BEG] . . . 12
1.19 Disadvantages [INT] . . . 12

1.20 When Should We Use It? [BEG] . . . 12
1.21 When Should We NOT Use It? [INT] . . . 13
1.22 Common Mistakes [INT] . . . 13
1.23 Interview Questions & Answers . . . 13
Scenario-Based Question . . . 14
Tricky Question . . . 14
1.24 Comparison with Similar Patterns . . . 14
1.25 Small Coding Task for Practice . . . 14
1.26 Expected Interview Answer (30 seconds) . . . 14
1.27 Expected Interview Answer (2 minutes) . . . 14

2. Factory Method Pattern [INT] 16
2.1 Problem with Direct Object Creation . . . 16
2.2 Simple Factory (Not a GoF pattern, but a common first step) [BEG] . . . 16
2.3 Factory Method Pattern (The Real GoF Pattern) [ADV] . . . 16
2.4 Factory Method vs Simple Factory [INT] (Common Confusion) . . . 18
2.5 Class Relationship Diagram . . . 18
2.6 Real-World Examples . . . 18
2.7 Backend Development Example . . . 18
2.8 Spring Boot Connection [ADV] . . . 18
2.9 Advantages [BEG] . . . 19
2.10 Disadvantages [INT] . . . 19
2.11 When to Use [BEG] . . . 19
2.12 When NOT to Use [INT] . . . 19
2.13 Common Mistakes [INT] . . . 19
2.14 Interview Questions . . . 20
Scenario-Based Question . . . 20
2.15 Small Coding Task . . . 20
2.16 Expected Interview Answer (30 seconds) . . . 20
2.17 Expected Interview Answer (2 minutes) . . . 20

What's Next? 22

PART 1 — DESIGN PATTERN FUNDAMENTALS

1. What is a Design Pattern? [BEG]

A **Design Pattern** is a **proven, reusable solution to a commonly occurring problem** in software design. It is NOT code you copy-paste — it is a **template / blueprint** of how to solve a problem, which you adapt to your own class names and logic.

Think of it like a **recipe**. A recipe tells you steps to make a dish, but you still cook it yourself with your own ingredients. A Design Pattern tells you the *structure* of the solution, and you fill in your own classes.

Hinglish: Design pattern matlab "pehle se test kiya hua solution" jo experienced developers ne bohot saari real problems solve karte waqt discover kiya. Aap wheel dubara invent nahi karte, balki ek accepted approach follow karte ho.

2. Why were Design Patterns Introduced? [BEG]

Before patterns were documented, every developer solved the same recurring problems (object creation, notifying multiple parts of a system, wrapping objects with extra features) in their **own inconsistent way**. This caused:

- Code that was hard for other developers to understand (no shared vocabulary)
- Repeated mistakes that experienced developers had already solved before
- Difficult maintenance because there was no standard structure

In 1994, four authors — Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides — known as the **"Gang of Four" (GoF)** — published a book documenting **23 common solutions**. This gave the industry a **shared vocabulary**.

3. What Problem Do Design Patterns Solve? [BEG]

Without Patterns	With Patterns
Every developer invents their own structure	Common, well-understood structure
Hard to communicate design ("that weird class thing you did")	Shared vocabulary ("just use a Strategy here")
Code tightly coupled, hard to extend	Code loosely coupled, extensible
Repeating same mistakes	Reusing proven solutions
Hard to onboard new developers	New developers instantly recognize the pattern

4. Design Pattern vs Framework [BEG]

Design Pattern	Framework
A concept / idea / template	Actual reusable code (a library you plug into)
You write the code yourself	Framework provides ready code, you just configure/extend
Example: Singleton pattern	Example: Spring Framework
No inversion of control	Framework usually calls your code (Inversion of Control)

Interview trap: People confuse "Spring uses Singleton" with "Spring IS a Singleton pattern." Spring is a framework that *internally applies* several design patterns.

5. Design Pattern vs Library [BEG]

- A **library** is a set of ready classes/methods you call (e.g., Apache Commons, Gson).
- A **pattern** is not code at all — it's an approach you can implement using plain Java, with no library needed.

6. Design Pattern vs Algorithm [BEG]

- An **algorithm** solves a **computational** problem (e.g., sorting, searching) with defined steps and a predictable output.
- A **pattern** solves a **structural/design** problem (how classes/objects relate to each other), not a computation.

7. Design Pattern vs Architecture [INT]

- **Architecture** is the big picture — how the entire system is organized (e.g., Microservices, Layered Architecture, MVC, Event-Driven Architecture).
- **Design Pattern** operates at a much smaller scale — how a handful of classes interact to solve one specific problem.

Think of Architecture as the "city plan" and Design Patterns as the "individual building designs" inside that city.

8. Advantages of Design Patterns [BEG]

1. Proven solutions — reduce risk of design mistakes
2. Common vocabulary for team communication
3. Easier to maintain and extend code
4. Encourage loose coupling and high cohesion
5. Make code more testable
6. Speed up development (less reinventing the wheel)

9. Disadvantages of Design Patterns [INT]

1. **Overengineering** — applying patterns where a simple solution would do
2. Increases number of classes/interfaces → can reduce readability for simple problems
3. Learning curve for junior developers
4. Misuse when developers force-fit a pattern just to "look smart"
5. Can add indirection that makes debugging harder if overused

Golden Rule: *"Patterns should emerge from the problem. Don't start with a pattern and search for a problem to justify it."*

10. What is GoF? [BEG]

GoF = Gang of Four. The book *"Design Patterns: Elements of Reusable Object-Oriented Software"* (1994) by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. It documented **23 patterns**, which remain the foundation of design pattern interviews today.

11. The 23 GoF Design Patterns [BEG]

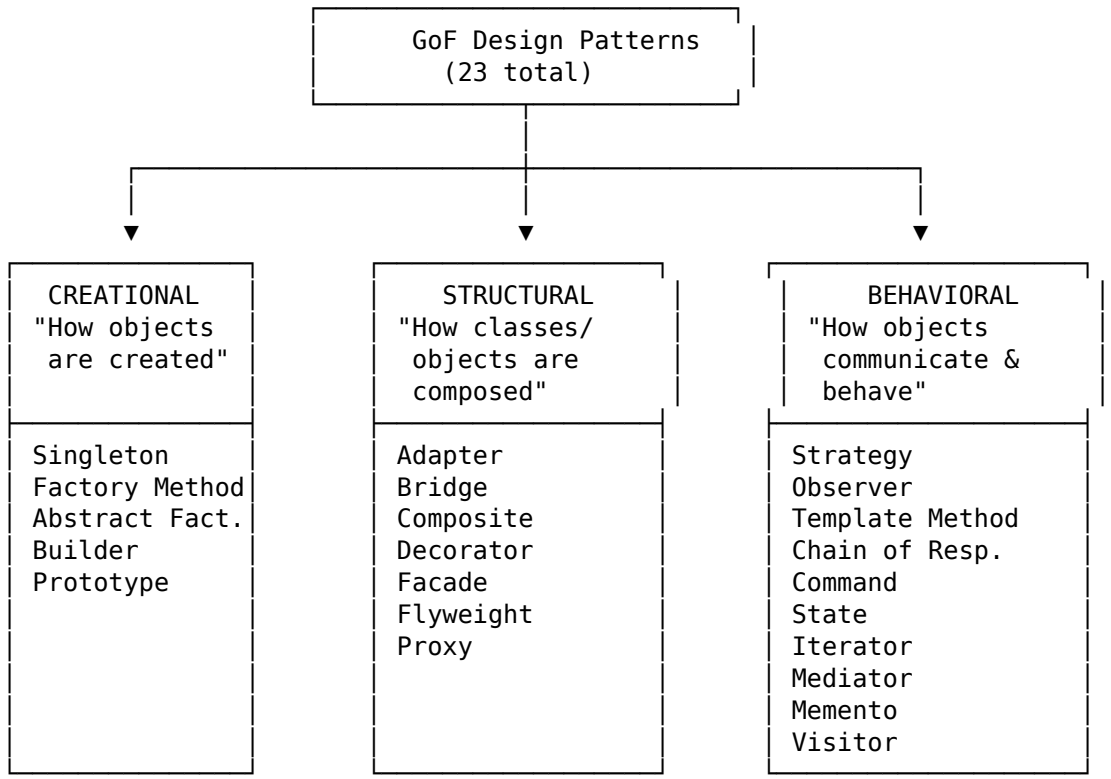
Creational (5): Singleton, Factory Method, Abstract Factory, Builder, Prototype

Structural (7): Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

Behavioral (11): Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

(Note: Interpreter pattern is rarely asked in Java backend interviews, so we will keep focus on the other 22 practically.)

12. The Three Major Categories [BEG]



Simple memory trick: - **Creational** = “Kaise banaye” (How to create) - **Structural** = “Kaise jode” (How to connect/compose) - **Behavioral** = “Kaise behave kare / baat kare” (How objects interact)

13. How Design Patterns Relate to OOP [INT]

Design patterns are built entirely on the 4 pillars of OOP:

- **Encapsulation** → Builder hides construction complexity; Singleton hides instance creation
- **Abstraction** → Strategy/Template Method define abstract steps
- **Inheritance** → Template Method relies on it heavily
- **Polymorphism** → Strategy, Factory, Observer all rely on interchangeable implementations via a common interface/superclass

Most GoF patterns follow the principle: **“Program to an interface, not an implementation.”**

14. Design Patterns vs SOLID Principles [ADV]

SOLID principles are **rules/goals**. Design patterns are **techniques that help you follow those rules**.

SOLID Principle	Pattern(s) that help achieve it
Single Responsibility	Facade (separates orchestration from business logic), Builder (separates construction from representation)
Open/Closed	Strategy, Decorator, Factory Method (add new behavior without modifying existing code)
Liskov Substitution	Template Method, Strategy (subtypes must be substitutable)
Interface Segregation	Adapter, Strategy (small, focused interfaces)
Dependency Inversion	Factory, Strategy, Dependency Injection (Spring) (depend on abstractions, not concrete classes)

15. Can Design Patterns Violate SOLID? [ADV] (Tricky Interview Question)

Yes, if misused. Example: A bloated Singleton that does logging + caching + configuration + DB access violates **Single Responsibility Principle** — this is called the **"God Object"** anti-pattern. A pattern is only good when applied correctly; blind application can *break* SOLID instead of supporting it.

16. Should Every Application Use Design Patterns? [INT] (Common Interview Question)

No. Use a pattern only when: - The problem actually recurs (not just "might happen someday") - Simpler code would clearly become messy without it - The team understands the pattern (avoid confusing teammates)

Forcing patterns into a small CRUD app just to show off patterns is a classic beginner mistake, often called **"Pattern-itis"** or **over-engineering**.

> **30-second answer**: "Design patterns are proven, reusable solutions to common object-oriented design problems, first documented by the Gang of Four in 1994. They're grouped into Creational, Structural, and Behavioral categories, and they help write flexible, maintainable code that follows SOLID principles — but they should be applied only when the problem genuinely calls for them, not by default."

PART 2 — CREATIONAL DESIGN PATTERNS

1. Singleton Pattern [BEG]

1.1 Definition

Singleton ensures a class has **only ONE instance** in the entire JVM, and provides a **global access point** to that instance.

1.2 The Problem (before Singleton)

Imagine a `ConfigManager` class that reads a config file. If every part of the application does `new ConfigManager()`, you get: - Multiple objects reading the same file redundantly (wasted memory/IO) - Inconsistent state — one part updates a value, another instance doesn't see it - No single "source of truth"

```
// BAD: everyone creates their own instance
ConfigManager c1 = new ConfigManager(); // reads file
ConfigManager c2 = new ConfigManager(); // reads file AGAIN
// c1 and c2 are different objects — if c1 changes a value, c2 doesn't know
```

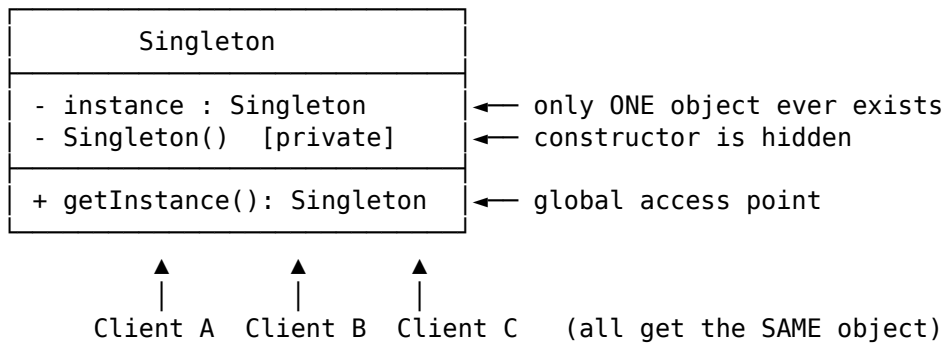
1.3 Why Singleton? — What It Solves

- Guarantees a **single shared instance** — one source of truth
- Saves memory/resources (only one DB connection pool, one logger, one config object)
- Provides a **controlled, global access point**

1.4 Step-by-Step Working

1. Make the **constructor `private`** so nobody outside the class can call `new`.
2. Store the single instance in a `private static` field inside the class itself.
3. Provide a `public static getInstance()` method that returns that one instance (creating it the first time, if lazy).
4. Every caller uses `ClassName.getInstance()` instead of `new ClassName()`.

1.5 Class Relationship Diagram



1.6 Eager Initialization Singleton

```
public class EagerConfigManager {
    // Instance created immediately when class is loaded — before anyone even asks for it
    private static final EagerConfigManager INSTANCE = new EagerConfigManager();
```

```
    // Private constructor — nobody outside can do "new EagerConfigManager()"
    private EagerConfigManager() {
        System.out.println("Loading configuration from file...");
    }

    // Public static method — the only door to get the object
    public static EagerConfigManager getInstance() {
        return INSTANCE;
    }
}
```

Line-by-line: - `private static final EagerConfigManager INSTANCE = new EagerConfigManager();` → JVM creates this object the moment the class is loaded into memory, only once, guaranteed thread-safe by the JVM's class-loading mechanism. - `private EagerConfigManager()` → blocks any external `new` call — this is the core trick of Singleton. - `getInstance()` → simply hands back the already-created object.

Downside: object is created even if it's never used — wastes memory if the object is heavy and rarely needed.

1.7 Lazy Initialization Singleton (Not Thread-Safe)

```
public class LazyConfigManager {
    private static LazyConfigManager instance;

    private LazyConfigManager() { }

    public static LazyConfigManager getInstance() {
        if (instance == null) {           // check 1
            instance = new LazyConfigManager(); // create only when first needed
        }
        return instance;
    }
}
```

Problem: If **two threads** call `getInstance()` at the exact same time, BOTH can pass the `if (instance == null)` check before either finishes creating the object → **two instances get created**, breaking the Singleton guarantee. This is a classic **race condition**.

1.8 Thread-Safe Singleton — `synchronized` method [INT]

```
public class ThreadSafeConfigManager {
    private static ThreadSafeConfigManager instance;

    private ThreadSafeConfigManager() { }

    public static synchronized ThreadSafeConfigManager getInstance() {
        if (instance == null) {
            instance = new ThreadSafeConfigManager();
        }
        return instance;
    }
}
```

`synchronized` on the method means only **one thread at a time** can enter `getInstance()`. This fixes the race condition, but **every single call** — even after the instance already exists — has to wait for the lock. This is a **performance cost** because locking is expensive when called millions of times.

1.9 Double-Checked Locking + `volatile` [ADV] (Most Asked in Interviews)

```
public class DCLConfigManager {
    // volatile prevents instruction reordering & ensures visibility across threads
    private static volatile DCLConfigManager instance;

    private DCLConfigManager() { }

    public static DCLConfigManager getInstance() {
        if (instance == null) {                    // 1st check (no lock) — fast path
            synchronized (DCLConfigManager.class) { // lock only if instance might be null
                if (instance == null) {             // 2nd check (inside lock) — safety
                    instance = new DCLConfigManager();
                }
            }
        }
        return instance;
    }
}
```

Why two checks? - **1st check (no lock)**: After the instance is created once, all future calls skip the expensive `synchronized` block entirely — huge performance win. - **2nd check (inside lock)**: Protects against the case where two threads both passed the 1st check before the instance existed — only one of them actually creates the object; the other sees `instance != null` inside the lock and skips creation.

Why `volatile` is critical: `instance = new DCLConfigManager()` is NOT one atomic step internally. The JVM may: 1. Allocate memory for the object 2. Assign the reference to `instance` 3. Run the constructor to initialize fields

Without `volatile`, the JVM/CPU is allowed to **reorder steps 2 and 3**. Another thread could see a **non-null but half-initialized object** (reference assigned, constructor not finished) and start using broken data. `volatile` prevents this reordering and guarantees visibility of fully-constructed objects across threads.

1.10 Enum Singleton [ADV] (Best Practice — Josh Bloch recommended)

```
public enum ConfigManagerEnum {
    INSTANCE;

    private final Map<String, String> settings = new HashMap<>();

    public String get(String key) {
        return settings.get(key);
    }

    public void set(String key, String value) {
        settings.put(key, value);
    }
}
```

```
// Usage:
ConfigManagerEnum.INSTANCE.set("env", "production");
String env = ConfigManagerEnum.INSTANCE.get("env");
```

Why this is considered the BEST Singleton implementation: 1. **Thread-safe by default** — JVM guarantees enum constants are initialized only once, safely, even under multi-threading. 2. **Serialization-safe automatically** — normal Singletons can be broken by Java serialization (deserializing creates a *new* object!). Enums have built-in serialization support that preserves the single instance. 3. **Reflection-proof** — normal Singletons can be broken via reflection (`constructor.setAccessible(true)` can call the private constructor anyway!). Java explicitly **forbids** reflection from instantiating enum constants — throws `IllegalArgumentException`.

1.11 The Serialization Problem [ADV]

```
// If ThreadSafeConfigManager implements Serializable:
ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("config.ser"));
out.writeObject(instance1);

ObjectInputStream in = new ObjectInputStream(new FileInputStream("config.ser"));
ThreadSafeConfigManager instance2 = (ThreadSafeConfigManager) in.readObject();

System.out.println(instance1 == instance2); // false! Singleton is BROKEN
```

Fix: Add this method — Java calls it automatically during deserialization:

```
protected Object readResolve() {
    return instance; // return the existing instance instead of the new deserialized one
}
```

1.12 The Reflection Problem [ADV]

```
Constructor<ThreadSafeConfigManager> constructor =
        ThreadSafeConfigManager.class.getDeclaredConstructor();
constructor.setAccessible(true);          // bypasses "private"
ThreadSafeConfigManager hacked = constructor.newInstance(); // creates a 2nd instance!
```

Fix (partial): Throw an exception inside the constructor if an instance already exists:

```
private ThreadSafeConfigManager() {
    if (instance != null) {
        throw new IllegalStateException("Instance already exists — use getInstance()");
    }
}
```

Best fix: Use **Enum Singleton** — reflection literally cannot bypass it.

1.13 The Cloning Problem [INT]

If the Singleton class implements `Cloneable`, calling `.clone()` creates a **second object** without going through the constructor at all.

Fix: Override `clone()` to throw an exception:

```
@Override
protected Object clone() throws CloneNotSupportedException {
```

```
    throw new CloneNotSupportedException("Cloning not allowed for Singleton");
}
```

1.14 Singleton vs Static Class [INT] (Very Common Interview Question)

Singleton	Static Class
Is a real object (instance)	No object at all — only static methods/fields
Can implement interfaces	Cannot implement interfaces meaningfully (static methods aren't polymorphic)
Can be passed around as a parameter (`SomeInterface obj = Singleton.getInstance();`)	Cannot be passed as a parameter/reference
Can be lazily created	Loaded when class is first referenced
Supports inheritance & polymorphism	No inheritance/polymorphism benefit
Easier to mock in unit tests (via interface)	Very hard to mock/unit test
Can maintain instance state across calls in an OOP way	Also holds state, but no object identity

Interview trick answer: "A static class is a language-level restriction with no object identity and no polymorphism, while a Singleton is a genuine object that happens to be limited to one instance — so it can implement interfaces, be injected as a dependency, and be mocked in tests. That's why frameworks like Spring prefer the Singleton *pattern* concept over static utility classes for stateful shared components."

1.15 Singleton in Spring Boot [ADV] (Very Important — Common Confusion)

```
@Service
public class PaymentService {
    // Spring creates ONE instance of this bean per ApplicationContext by default
}
```

CRITICAL DISTINCTION (frequently asked):

GoF Singleton	Spring "Singleton" Bean Scope
ONE instance per **entire JVM**	ONE instance per **Spring ApplicationContext** (you could have 2 contexts in one JVM in rare cases → 2 "singleton" beans exist)
Enforced via `private` constructor + `getInstance()`	Enforced by the **container**, NOT by the class itself — you can still do `new PaymentService()` yourself if you wanted (bad practice, but compiles fine)
Class-level guarantee	Container-level/configuration-level guarantee
Hard-coded, cannot be changed	Can change scope to `prototype`, `request`, `session` with `@Scope` annotation

Interview one-liner: "Spring's singleton scope is a *container-managed convention* — Spring guarantees only one bean instance per context, but the class itself has a normal public constructor. GoF Singleton is a *self-enforced* guarantee at the class level using a private constructor."

1.16 Real-World Examples

- `java.lang.Runtime.getRuntime()` — JVM runtime environment, one per JVM
- Logger instances (Log4j `Logger`, though usually one per class, not one per app)
- Database connection pool managers (e.g., HikariCP's pool object)
- Spring `ApplicationContext` itself
- `java.awt.Desktop.getDesktop()`

1.17 Best Implementation in Modern Java (Java 17/21) [ADV]

Recommended answer for interviews: Use **Enum Singleton** for simple cases (no lazy loading needed), or **Double-Checked Locking with `volatile`** when you need lazy initialization with fine control (e.g., needing constructor parameters or extending a class, which enums can't do since they can't extend other classes).

```
public enum DatabaseConnectionManager {
    INSTANCE;

    private final DataSource dataSource;

    DatabaseConnectionManager() {
        this.dataSource = createDataSource(); // heavy setup done once, safely
    }

    private DataSource createDataSource() {
        // configure HikariCP or similar
        return null; // placeholder
    }

    public DataSource getDataSource() {
        return dataSource;
    }
}
```

1.18 Advantages [BEG]

- Controlled access to a single instance
- Saves memory for expensive objects (DB pools, caches)
- Global, consistent state across the application

1.19 Disadvantages [INT]

- Makes **unit testing harder** (global state, hidden dependencies, hard to mock without DI)
- Can become a **hidden dependency** — code that secretly relies on global state is hard to trace
- Violates **Single Responsibility** if the Singleton starts doing too much
- Can be a bottleneck if `synchronized` is overused

1.20 When Should We Use It? [BEG]

- Exactly one instance is logically required (config manager, connection pool, cache manager, logging facility)
- Shared state truly needs to be global and consistent

1.21 When Should We NOT Use It? [INT]

- When the object should be different per request/user (e.g., in Spring, `@Scope("prototype")` or `@Scope("request")` is more appropriate)
- When it makes unit testing painful — prefer Dependency Injection instead of hard-coded `getInstance()` calls
- When you're using Spring — let Spring manage the singleton via `@Component`/`@Service` instead of writing GoF Singleton boilerplate yourself

1.22 Common Mistakes [INT]

1. Forgetting `volatile` in double-checked locking → subtle multithreading bugs
2. Making the Singleton hold **too much responsibility** (God Object anti-pattern)
3. Using GoF Singleton pattern manually inside a Spring app instead of letting Spring manage it (`@Component` already gives you singleton scope — writing `getInstance()` too is redundant and conflicts with DI)
4. Forgetting to handle serialization/reflection/cloning attacks when the interviewer probes deeper
5. Using Singleton for objects that actually need per-request/per-user state → causes data leaking between users (dangerous bug in web apps!)

1.23 Interview Questions & Answers

Q1: Why is the Singleton constructor private? > **Short answer**: To prevent any code outside the class from creating additional instances using `new`. > **Detailed answer**: If the constructor were public, any class could call `new Singleton()` and create multiple objects, defeating the entire purpose of the pattern. A private constructor forces all object creation to go through the single, controlled `getInstance()` method. > **Cross-question**: "But reflection can still call a private constructor — so is Singleton really enforced?" > **Best response**: "True — reflection can bypass a private constructor using `setAccessible(true)`. That's why for critical singletons, we either add a guard in the constructor that throws an exception if an instance already exists, or better, use Enum Singleton, which the JVM explicitly protects from reflective instantiation."

Q2: Can Singleton be broken? > Yes — via **Reflection**, **Serialization/Deserialization**, and **Cloning**. Each has a specific fix (constructor guard, `readResolve()`, overriding `clone()` to throw an exception). Enum Singleton is immune to all three by design.

Q3: Is Singleton thread-safe? > Depends on implementation. Eager initialization is thread-safe (JVM class-loading guarantee). Basic lazy initialization is NOT thread-safe. `synchronized` method is thread-safe but slow. Double-checked locking with `volatile` is thread-safe AND fast. Enum is thread-safe by JVM design.

Q4: Which Singleton implementation is best? > Enum Singleton for most cases (safest, simplest, handles serialization/reflection automatically). Use Double-Checked Locking when you need lazy loading with constructor logic that enums can't support (e.g., extending another class).

Q5: Why use `volatile` in double-checked locking? > To prevent instruction reordering during object construction, ensuring no thread ever sees a partially-constructed object, and to guarantee visibility of the fully-initialized object across all threads' caches.

Q6: Enum Singleton vs normal Singleton? > Enum is thread-safe, serialization-safe, and reflection-safe automatically, with far less boilerplate. Normal Singleton needs manual handling for all three, and is more flexible (can extend a class, can be lazily/eagerly controlled explicitly) — Enums cannot extend another class since they already implicitly extend `java.lang.Enum`.

Q7: Is Spring Bean Singleton the same as GoF Singleton? > No. Spring's singleton scope is a **container-level guarantee** (one bean instance per `ApplicationContext`), enforced by the framework, not the class. GoF Singleton is a **class-level guarantee** enforced with a private

constructor. A Spring `@Service` class still has a normal constructor and *could* technically be instantiated with `new` outside Spring's control.

Scenario-Based Question

"You're asked to design a centralized application configuration loader that reads a YAML file once at startup and is accessed from 50+ places in the app. How would you design it, and would you use GoF Singleton or Spring's singleton bean?"

> **Ideal answer**: "In a Spring Boot application, I would NOT hand-write a GoF Singleton with `getInstance()`. Instead, I'd create a `@Component` (or `@ConfigurationProperties` class) — Spring already manages it as a singleton bean within the application context, and I get dependency injection, easy testability with mocks, and lifecycle management for free. I would reserve manual GoF Singleton implementation for non-Spring, plain Java utilities where no DI container is available."

Tricky Question

"If I mark a class's constructor as private but do NOT provide a static `getInstance()` method, is it still usable?" > No — it becomes **impossible to instantiate at all** (unless via a static factory method, inner class, or reflection). This is actually how **Utility classes** (like `java.lang.Math`) prevent instantiation entirely — they have a private constructor and no `getInstance()`, because they don't need any instance at all (all methods are static).

1.24 Comparison with Similar Patterns

Singleton vs Static Class — covered above in 1.14 in full detail.

1.25 Small Coding Task for Practice

> **Task**: Implement a `LoggerService` Singleton using Double-Checked Locking. It should have a method `log(String message)` that appends timestamped log lines to an in-memory `List<String>`, and a method `getLogs()` returning all collected logs. Ensure it's thread-safe and cannot be broken via reflection (add a constructor guard).

1.26 Expected Interview Answer (30 seconds)

> "Singleton ensures a class has only one instance across the JVM and provides a global access point to it, using a private constructor and a static `getInstance()` method. It's commonly implemented via Enum Singleton for safety, or double-checked locking with `volatile` for lazy, thread-safe initialization. It's used for shared resources like config managers, loggers, and connection pools — but in Spring, we usually rely on the container's singleton bean scope instead of writing it manually."

1.27 Expected Interview Answer (2 minutes)

> "Singleton is a creational design pattern that restricts a class to exactly one instance and provides a single global access point to it. The problem it solves is uncontrolled object creation — if multiple parts of an app each create their own instance of something like a configuration manager or connection pool, you get wasted resources and inconsistent state.
>
> We implement it by making the constructor private, storing the instance in a private static field, and exposing a public static `getInstance()` method. There are several variants: Eager initialization creates the instance at class-loading time — simple but wastes

memory if unused. Lazy initialization creates it on first use but isn't thread-safe by default. We fix that with a `synchronized` method, though that hurts performance on every call. The better fix is Double-Checked Locking combined with a `volatile` field — this checks `null` once without a lock for speed, and only synchronizes the first time the instance is actually being created, while `volatile` prevents any thread from seeing a half-constructed object due to instruction reordering.

The safest implementation, recommended by Joshua Bloch, is Enum Singleton — the JVM guarantees it's thread-safe, and it's automatically protected against both serialization and reflection attacks, which can otherwise break a normal Singleton.

In Spring Boot, we rarely hand-write this pattern — Spring beans are singleton-scoped by default at the container level, which is a different, framework-enforced guarantee rather than the class-level one from GoF. So for Spring apps, we let the container manage the singleton and instead focus on dependency injection and testability."

2. Factory Method Pattern [INT]

2.1 Problem with Direct Object Creation

```
// BAD: client code directly decides which concrete class to create
PaymentMethod payment;
if (type.equals("UPI")) {
    payment = new UpiPayment();
} else if (type.equals("CREDIT_CARD")) {
    payment = new CreditCardPayment();
} else if (type.equals("PAYPAL")) {
    payment = new PayPalPayment();
} else {
    throw new IllegalArgumentException("Unknown payment type");
}
payment.pay(1000);
```

Problems with this approach: 1. Every place that needs a `PaymentMethod` repeats this same if-else/switch block (duplication). 2. Adding a new payment type (e.g., `CryptoPayment`) means **editing every single place** this if-else exists — violates the **Open/Closed Principle** (code should be open for extension, closed for modification). 3. Client code is tightly coupled to **concrete classes** (`UpiPayment`, `CreditCardPayment`), not abstractions.

2.2 Simple Factory (Not a GoF pattern, but a common first step) [BEG]

```
public class PaymentFactory {
    public static PaymentMethod create(String type) {
        return switch (type) {
            case "UPI" -> new UpiPayment();
            case "CREDIT_CARD" -> new CreditCardPayment();
            case "PAYPAL" -> new PayPalPayment();
            default -> throw new IllegalArgumentException("Unknown type: " + type);
        };
    }
}
```

This centralizes creation into **one place**, but the if-else/switch problem still technically exists — it's just moved into one class instead of scattered everywhere. This is called **Simple Factory** — a useful convention, but NOT one of the official 23 GoF patterns.

2.3 Factory Method Pattern (The Real GoF Pattern) [ADV]

Definition: Factory Method defines an **interface (or abstract method)** for creating an object, but lets **subclasses decide** which concrete class to instantiate.

```
// Step 1: Product interface
public interface PaymentMethod {
    void pay(double amount);
}

// Step 2: Concrete Products
public class UpiPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " via UPI");
    }
```

```
}

public class CreditCardPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " via Credit Card");
    }
}

public class PayPalPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " via PayPal");
    }
}

// Step 3: Creator (abstract class declares the "factory method")
public abstract class PaymentProcessor {

    // This IS the Factory Method — subclasses decide what to create
    protected abstract PaymentMethod createPaymentMethod();

    // Common workflow that uses the created object — this is the real value!
    public void processPayment(double amount) {
        PaymentMethod method = createPaymentMethod(); // delegated to subclass
        System.out.println("Validating payment of ₹" + amount + "...");
        method.pay(amount);
        System.out.println("Payment processed successfully.");
    }
}

// Step 4: Concrete Creators — each decides which product to build
public class UpiPaymentProcessor extends PaymentProcessor {
    protected PaymentMethod createPaymentMethod() {
        return new UpiPayment();
    }
}

public class CreditCardPaymentProcessor extends PaymentProcessor {
    protected PaymentMethod createPaymentMethod() {
        return new CreditCardPayment();
    }
}
```

Usage:

```
PaymentProcessor processor = new UpiPaymentProcessor();
processor.processPayment(500.0);
// Output:
// Validating payment of ₹500.0...
// Paid ₹500.0 via UPI
// Payment processed successfully.
```

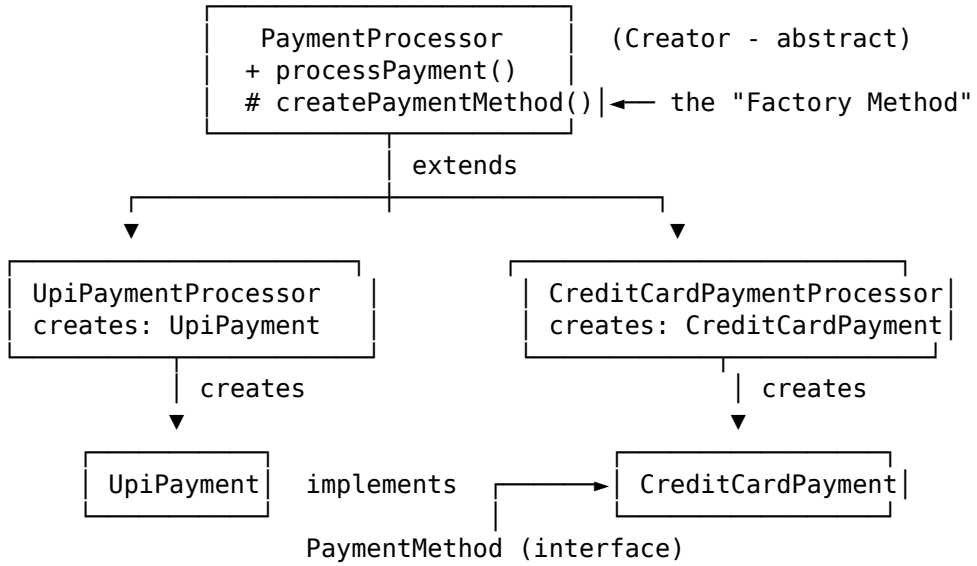
Line-by-line explanation: - `PaymentMethod` interface → the **Product** — what gets created. - `UpiPayment`, `CreditCardPayment` → **Concrete Products** — actual implementations. - `PaymentProcessor` (abstract class) → the **Creator** — declares the factory method `createPaymentMethod()` as `abstract`, and contains the shared business logic (`processPayment`) that USES the product without knowing its concrete type. - `UpiPaymentProcessor` → **Concrete Creator** — overrides the factory

method to decide exactly which product gets built. - The key insight: `processPayment()` never says `new UpiPayment()` directly — it calls the abstract `createPaymentMethod()`, and polymorphism decides which concrete class actually gets instantiated at runtime.

2.4 Factory Method vs Simple Factory [INT] (Common Confusion)

Simple Factory	Factory Method (GoF)
One class with if-else/switch	Uses inheritance + abstract method, no if-else needed at the point of use
Not officially a GoF pattern	Official GoF pattern #1 in Creational category
Adding new type = editing the factory class	Adding new type = create a new subclass (Open/Closed compliant)
Simple, good for small apps	Slightly more classes, better for extensible systems

2.5 Class Relationship Diagram



2.6 Real-World Examples

- `java.util.Calendar.getInstance()` — returns different `Calendar` subclasses depending on locale
- `java.nio.charset.Charset.forName()`
- `javax.xml.parsers.DocumentBuilderFactory.newInstance()`
- Logging frameworks (`LoggerFactory.getLogger()` in SLF4J)

2.7 Backend Development Example

A notification system: `NotificationProcessor` (Creator) with `EmailProcessor`, `SmsProcessor`, `WhatsAppProcessor` subclasses each producing their own `Notifier` product — used when a backend must support pluggable channels without editing a central if-else.

2.8 Spring Boot Connection [ADV]

Spring itself uses Factory Method internally via **`FactoryBean<T>`**:

```java
@Component
public class PaymentMethodFactoryBean implements FactoryBean<PaymentMethod> {

    @Override
    public PaymentMethod getObject() {
        return new UpiPayment(); // Spring calls this to create the bean
    }

    @Override
    public Class<?> getObjectType() {
        return PaymentMethod.class;
    }
}
```

Important distinction: - "**Spring internally uses this concept**": every `@Bean` method in a `@Configuration` class is essentially a factory method — Spring calls it to obtain the object rather than calling `new` on your behalf directly through reflection sometimes, or via `FactoryBean` for more complex creation logic. - "**You should manually implement this pattern**": when you have a family of related products (payment types, notification channels, file exporters) and want each subclass to control which concrete product it builds, especially in domain/business logic layers, not just for Spring bean wiring.

2.9 Advantages [BEG]

- Removes duplicated if-else/switch logic from client code
- Follows Open/Closed Principle — add new product + new creator subclass without touching existing code
- Client code depends only on abstractions (`PaymentMethod`, `PaymentProcessor`)

2.10 Disadvantages [INT]

- Increases the number of classes (one Concrete Creator per Concrete Product)
- Can feel like overkill for a small, rarely-changing set of types
- Slightly harder to trace ("which subclass actually creates this?") for developers new to the codebase

2.11 When to Use [BEG]

- You have a family of related objects, and the exact type to create should be decided by a subclass or a runtime parameter
- You expect NEW types to be added often, and don't want to modify existing code each time

2.12 When NOT to Use [INT]

- Only 1–2 fixed types that will basically never change → a simple `if-else` or `switch` is fine and simpler to read
- When using Spring, and a simple `@Bean` or `@Qualifier`-based injection would be clearer than manual factory classes

2.13 Common Mistakes [INT]

1. Confusing **Simple Factory** with **Factory Method** in interviews (biggest mistake — interviewers specifically test this!)

2. Creating factory subclasses for every trivial type even when a simple enum-based switch would suffice
3. Forgetting that the real value of Factory Method is the **shared algorithm in the Creator** (`processPayment()`), not just object creation — many candidates only show the creation part and miss the reusable workflow

2.14 Interview Questions

Q1: How does Factory Pattern remove if-else logic? > It moves the decision of "which class to instantiate" either into a Simple Factory class (centralizing if-else in one place) or, for the true GoF Factory Method, into subclasses using polymorphism, so the client code never writes conditional creation logic at all — it just calls `processPayment()` on whichever concrete creator it holds.

Q2: Factory Method vs Simple Factory — what's the real difference? > Simple Factory uses one class with conditional logic and isn't an official GoF pattern. Factory Method uses inheritance: an abstract Creator declares an abstract factory method, and each Concrete Creator subclass decides what Concrete Product to instantiate — this fully removes conditionals and follows Open/Closed Principle.

Scenario-Based Question

"Your payment system currently supports UPI and Credit Card. Tomorrow, product wants to add Cryptocurrency payments without touching existing tested code. How do you design this?" > "I'd use Factory Method: define a `PaymentMethod` interface and a `PaymentProcessor` abstract class with the shared workflow. For crypto, I just add a `CryptoPayment` class implementing `PaymentMethod` and a `CryptoPaymentProcessor` extending `PaymentProcessor` — zero changes to existing, already-tested classes, satisfying the Open/Closed Principle."

2.15 Small Coding Task

> **Task**: Build a `ReportGenerator` Factory Method system supporting `PdfReport`, `ExcelReport`, and `CsvReport`. Each Concrete Creator should have a shared `generate()` method in the abstract `ReportCreator` that logs "Generating report..." before calling the abstract `createReport()` factory method.

2.16 Expected Interview Answer (30 seconds)

> "Factory Method defines an abstract method for creating an object but lets subclasses decide the concrete type. It removes if-else creation logic, follows Open/Closed Principle, and is different from Simple Factory, which just centralizes conditional logic in one class rather than using polymorphism. Spring uses this concept internally through `@Bean` factory methods and `FactoryBean`."

2.17 Expected Interview Answer (2 minutes)

> "The Factory Method pattern solves the problem of tightly-coupled, duplicated object-creation logic. Without it, code is littered with if-else or switch statements deciding which concrete class to instantiate — for example, choosing between `UpiPayment`, `CreditCardPayment`, or `PayPalPayment` based on a string. Every place needing this decision repeats the same conditional, and adding a new payment type means editing all of them, which violates the Open/Closed Principle.
>
> Factory Method fixes this using inheritance and polymorphism: we define a Product interface, like `PaymentMethod`, and an abstract Creator class, `PaymentProcessor`, which declares an abstract factory method, `createPaymentMethod()`, and contains the shared

business workflow, `processPayment()`, that uses the product without knowing its concrete type. Each Concrete Creator, like `UpiPaymentProcessor`, overrides the factory method to decide exactly which concrete product to build. Adding a new payment type just means adding a new product class and a new creator subclass — no existing code is touched.

It's important to distinguish this from Simple Factory, which is just a single class with an if-else or switch — that's a useful convention but not an official GoF pattern, and interviewers often test whether you know the difference. In Spring Boot, this concept appears through `@Bean` factory methods in `@Configuration` classes and the `FactoryBean` interface, where Spring itself calls a method to construct a bean instead of using `new` directly."

What's Next?

This is **Part 1** of the complete series. Coming up in the next parts:

- **Part 2**: Abstract Factory, Builder, Prototype (rest of Creational patterns)
- **Part 3**: Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight (Structural)
- **Part 4**: Strategy, Observer, Template Method, Chain of Responsibility, Command, State, Iterator, Mediator, Memento, Visitor (Behavioral)
- **Part 5**: Design Patterns in Spring Boot (deep mapping) + Full Real-World Project ("Online Payment & Order Management System")
- **Part 6**: All Comparison Tables + Interview Question Bank (200+ Qs) + Coding Practice Tasks
- **Part 7**: Final Cheat Sheet + 15-Day Study Roadmap

Ask for "Part 2" to continue.